

COSC3506/ITEC3506

Software Engineering

Module 8 Lesson 2

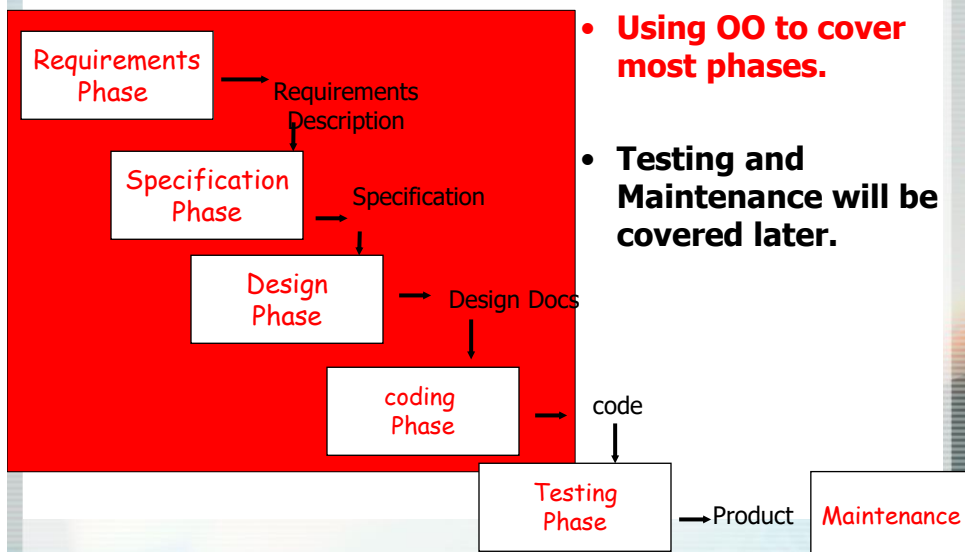
Implementation Model

Dr. Simon Xu

Copyright © 2020 by Dr. Simon Xu,

1

Where We Are?



2

Contents

- **Implementation Model**
- **Convert Design to Code: An Example**

Copyright © 2020 by Dr. Simon Xu,
CC BY-SA 4.0

3

Implementation Model

Copyright © 2020 by Dr. Simon Xu,
CC BY-SA 4.0

4

Implementation Model

What is Implementation Model?

- Including all the implementation artifacts, such as the **source code**, etc.... (what else??)

```

40 import java.awt.image.IndexColorModel;
41 import java.awt.image.ColorModel;
42 import java.awt.image.MemoryImageSource;
43 import java.awt.event.*;
44
45 /** The representation of a Chemical .xyz model */
46 class XYZChemModel {
47     float vert[];
48     Atom atoms[];
49     int tvert[];
50     int ZsortMap[];
51     int nvert, maxvert;
52
53     static Hashtable atomTable = new Hashtable();
54     static Atom defaultAtom;
55     static {
56         atomTable.put("C", new Atom(0, 0, 0));
57         atomTable.put("H", new Atom(210, 210, 210));
58         atomTable.put("N", new Atom(0, 0, 255));
59         atomTable.put("O", new Atom(255, 0, 0));
60     }
61 }

```

Copyright © 2020 by Dr. Simon Xu,

5

Mapping Designs to Code

How to conduct Implementation?

- class and interface definitions
- method definitions

```

class Point {
    int x,y;
    Color myColor;
    int currentX() {
        return x;
    }
    int currentY() {
        return y;
    }
    void move(int newX, int newY) {
        x = newX;
        y = newY;
    }
}

```

Annotations in the diagram:

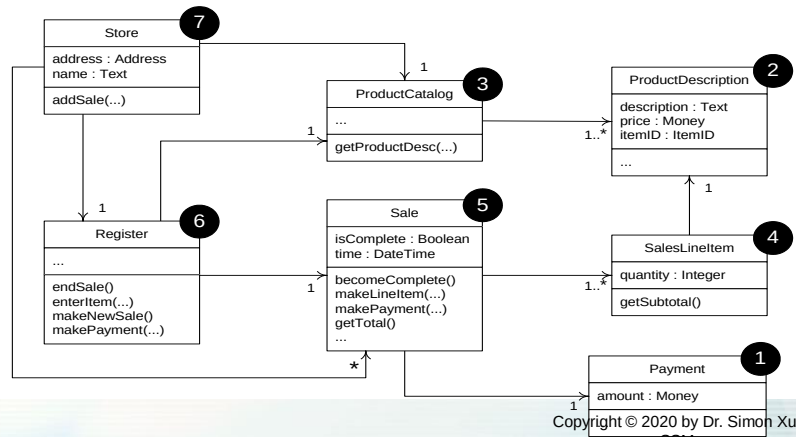
- instance variable declarations* (points to `int x,y;` and `Color myColor;`)
- method declarations* (points to `int currentX()`, `int currentY()`, and `void move()`)
- expression* (points to `x = newX;`)
- statement* (points to `y = newY;`)

Copyright © 2020 by Dr. Simon Xu,

6

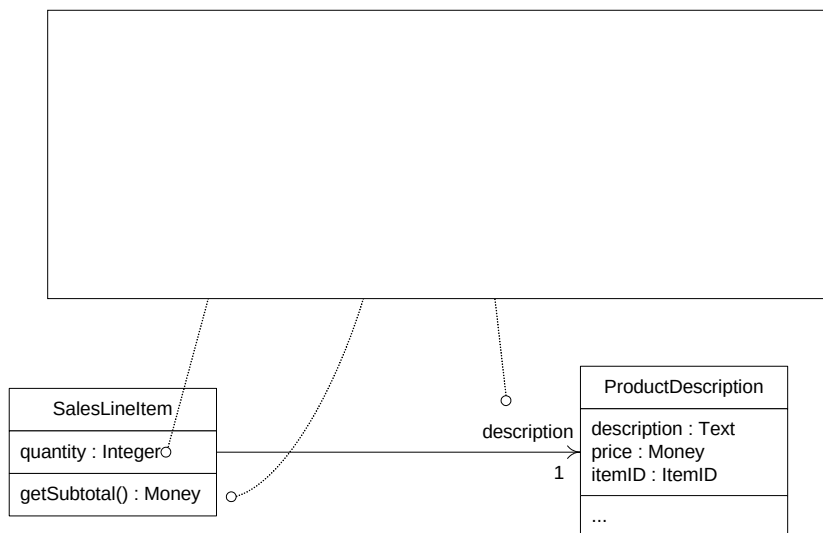
Creating Classes from DCDs

- DCDs depict the **class or interface name**, superclasses, **operation signatures**, and attributes of a class.



7

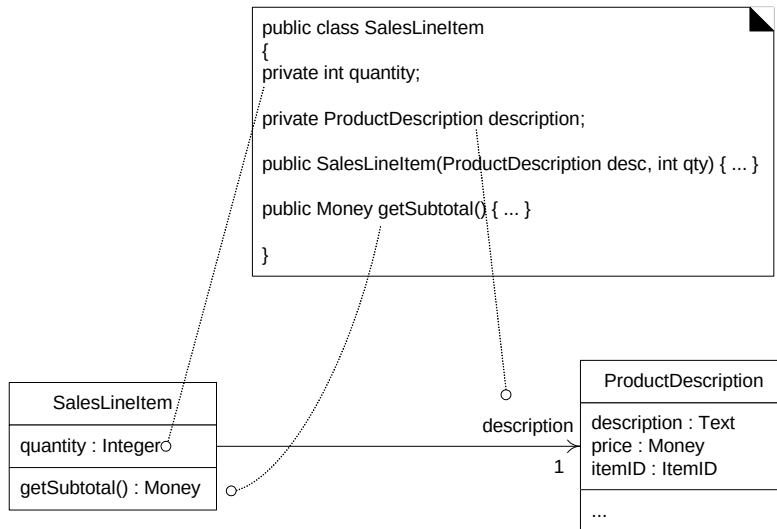
Creating Classes from DCDs



Copyright © 2020 by Dr. Simon Xu, CMU

8

Creating Classes from DCDs



Copyright © 2020 by Dr. Simon Xu,

9

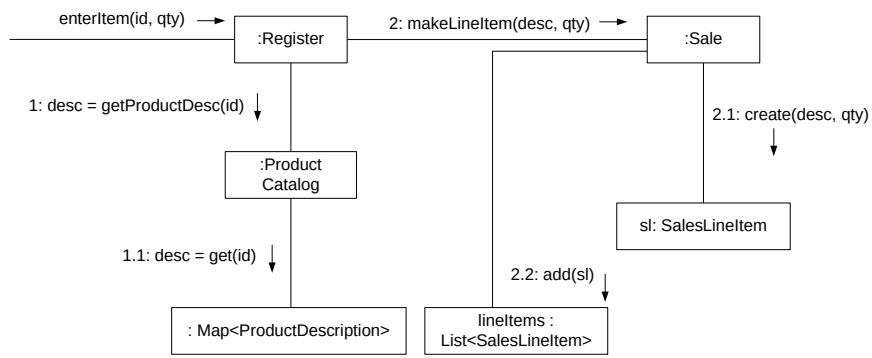
Creating Methods from Interaction Diagrams

- The sequence of the messages in an **interaction diagram** translates to a series of statements in the method definitions.
- The **enterItem** interaction diagram illustrates the Java definition of the **enterItem method**.
 - For the *Register* class.

Copyright © 2020 by Dr. Simon Xu,

10

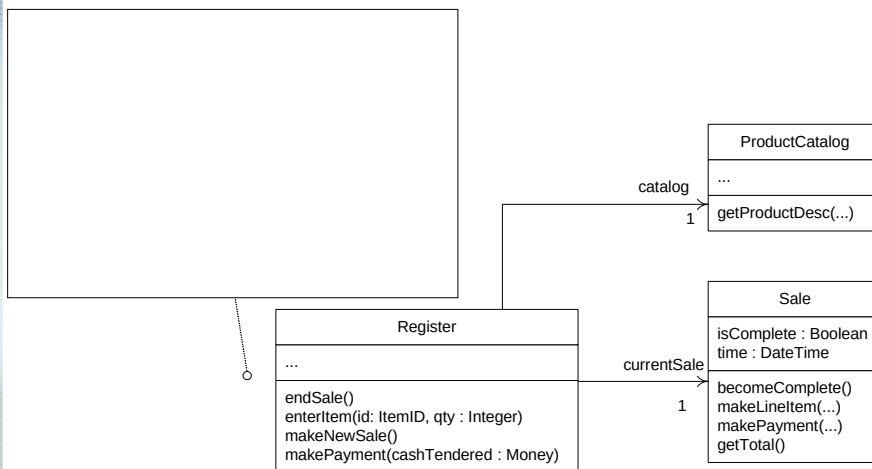
Creating Methods from Interaction Diagrams



Copyright © 2020 by Dr. Simon Xu,

11

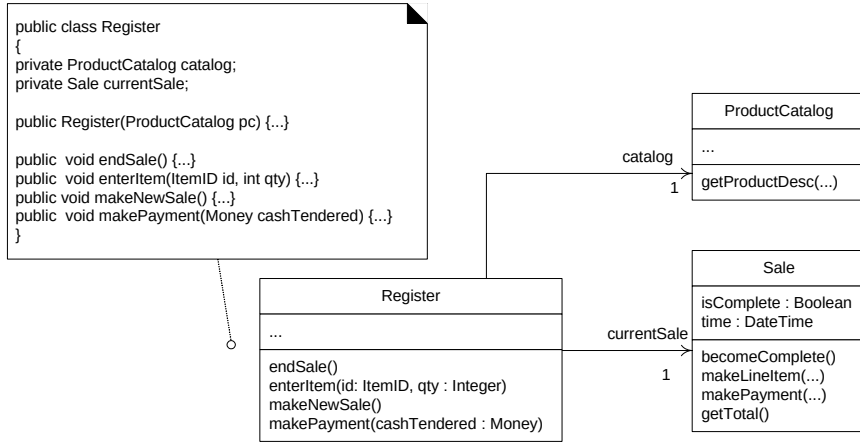
Creating Methods from Interaction Diagrams



Copyright © 2020 by Dr. Simon Xu,

12

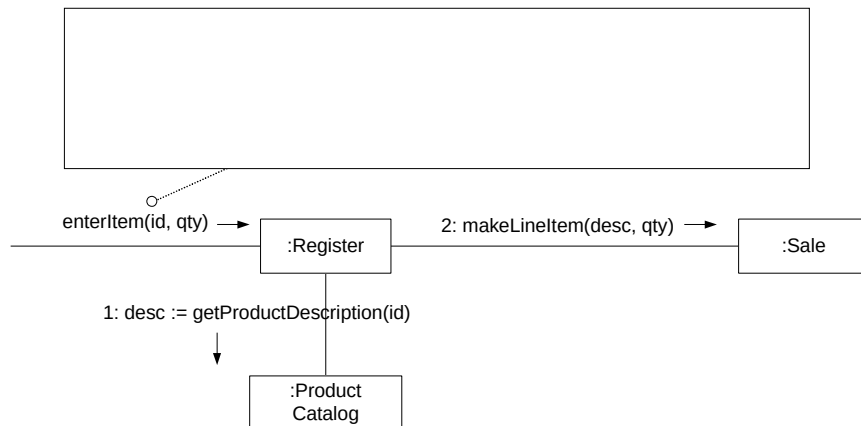
Creating Methods from Interaction Diagrams



Copyright © 2020 by Dr. Simon Xu,

13

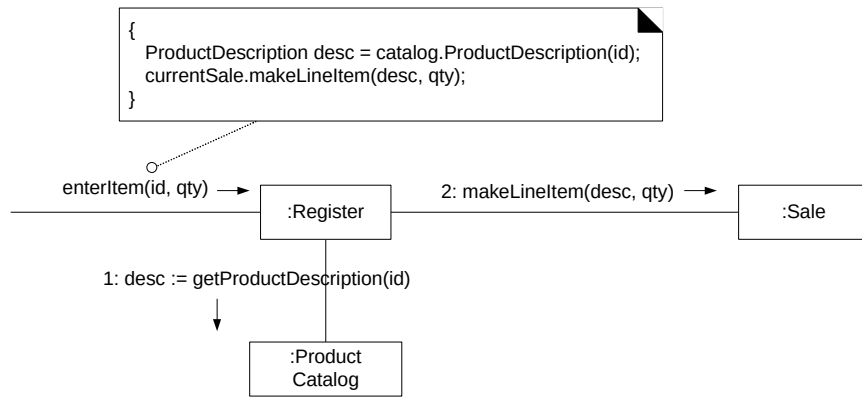
Creating Methods from Interaction Diagrams



Copyright © 2020 by Dr. Simon Xu,

14

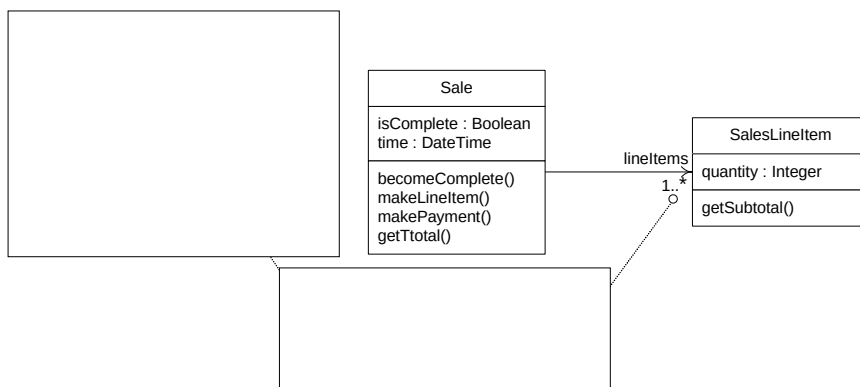
Creating Methods from Interaction Diagrams



Copyright © 2020 by Dr. Simon Xu,

15

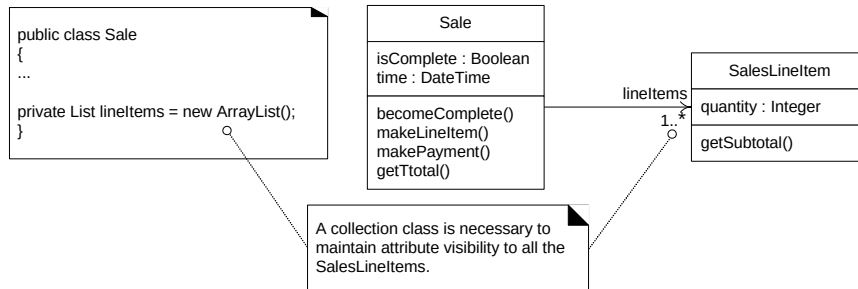
Collection Classes in Code



Copyright © 2020 by Dr. Simon Xu,

16

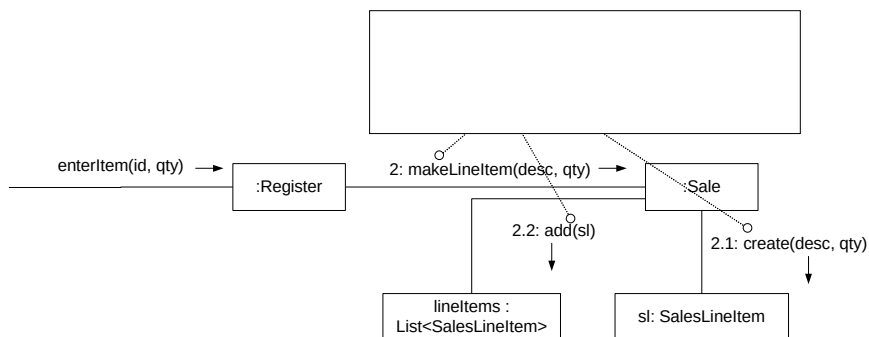
Collection Classes in Code



Copyright © 2020 by Dr. Simon Xu,

17

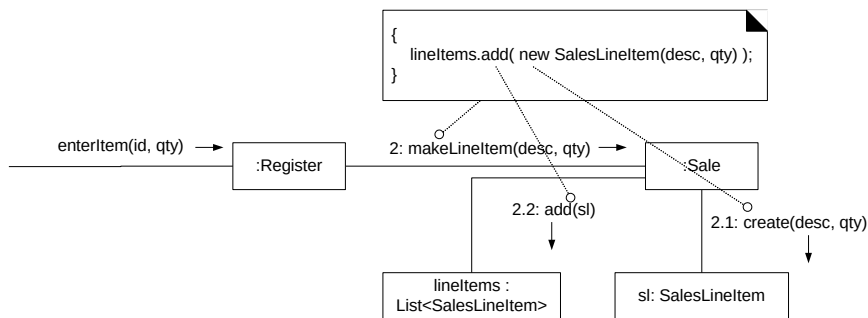
One final example (Sale.makeLineItem method)



Copyright © 2020 by Dr. Simon Xu,

18

One final example (Sale.makeLinItem method)



Copyright © 2020 by Dr. Simon Xu,

19

Summary

- As demonstrated, there is a **translation process**
 - from **UML class diagrams** to **class definitions**, and
 - from **interaction diagrams** to method bodies.
-
- The code example for the NextGen POS case study...
 - This code defines a simple case; it is not meant to illustrate a robust, fully developed Java program with synchronization, exception handling, and so on.

Copyright © 2020 by Dr. Simon Xu,

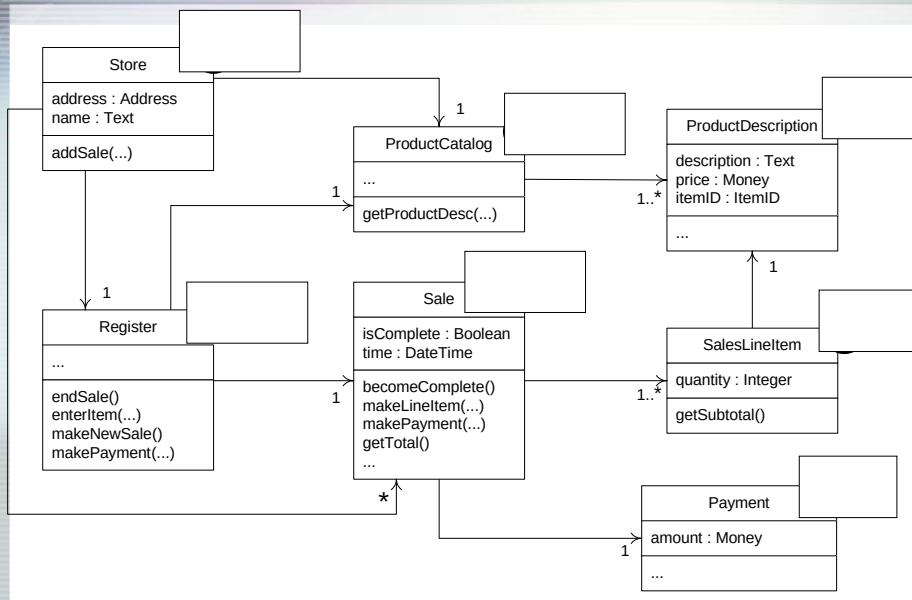
20

Convert Design to Code: An Example

Copyright © 2020 by Dr. Simon Xu,

21

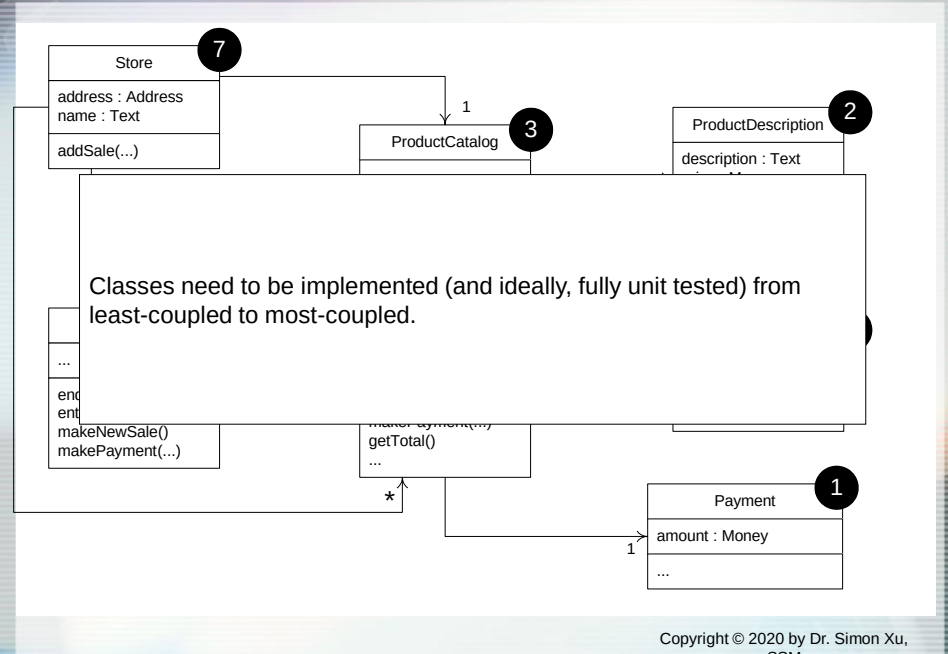
Order of Implementation



Copyright © 2020 by Dr. Simon Xu,

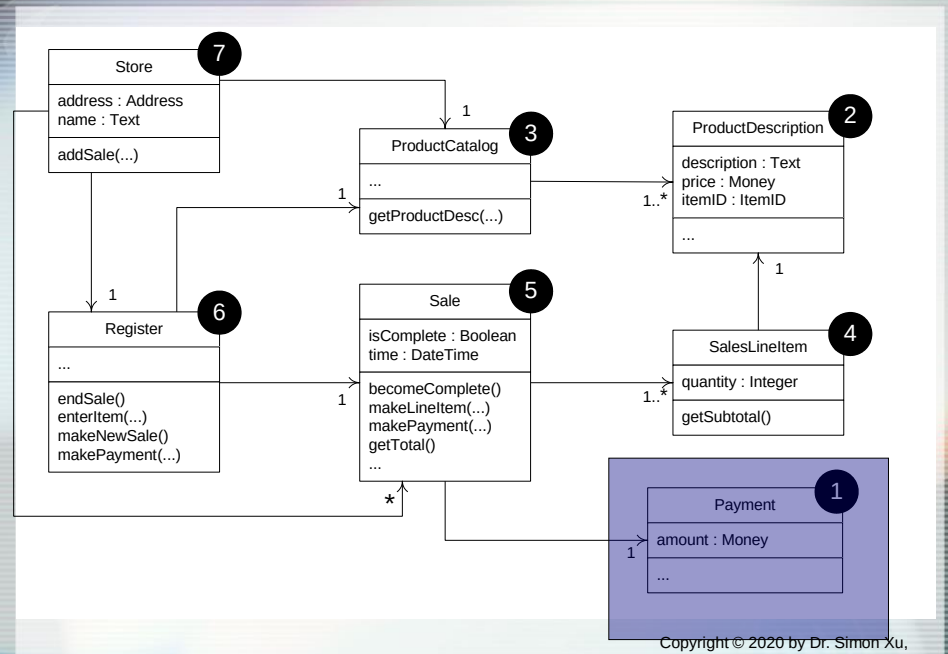
22

Order of Implementation



23

Order of Implementation



24

Class Payment

```
// all classes are probably in a package named
// something like:
package com.foo.nextgen.domain;

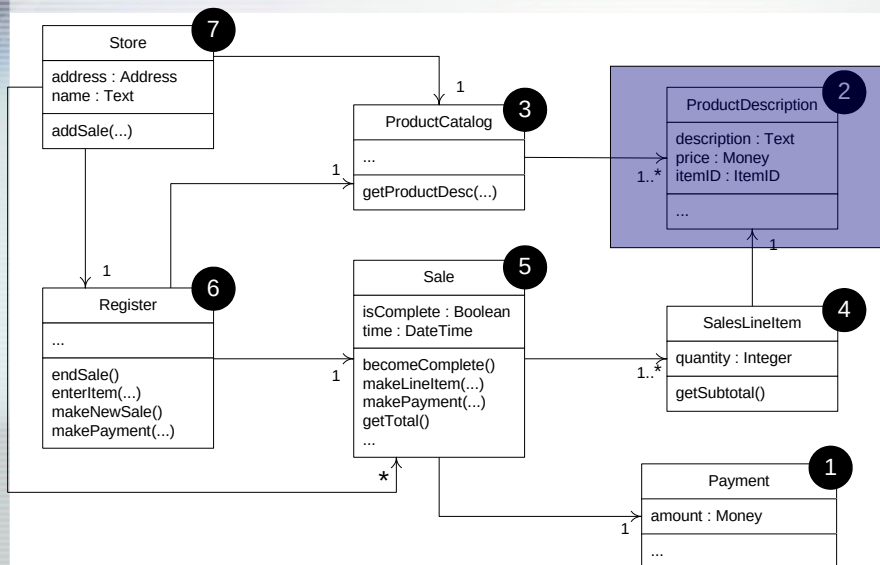
public class Payment
{
    private Money amount;

    public Payment( Money cashTendered ){ amount = cashTendered; }
    public Money getAmount() { return amount; }
}
```

Copyright © 2020 by Dr. Simon Xu,
CC BY-SA

25

Order of Implementation



Copyright © 2020 by Dr. Simon Xu,
CC BY-SA

26

Class ProductDescription

```
public class ProductDescription
{
    private ItemID id;
    private Money price;
    private String description;

    public ProductDescription
    ( ItemID id, Money price, String description )
    {
        this.id = id;
        this.price = price;
        this.description = description;
    }

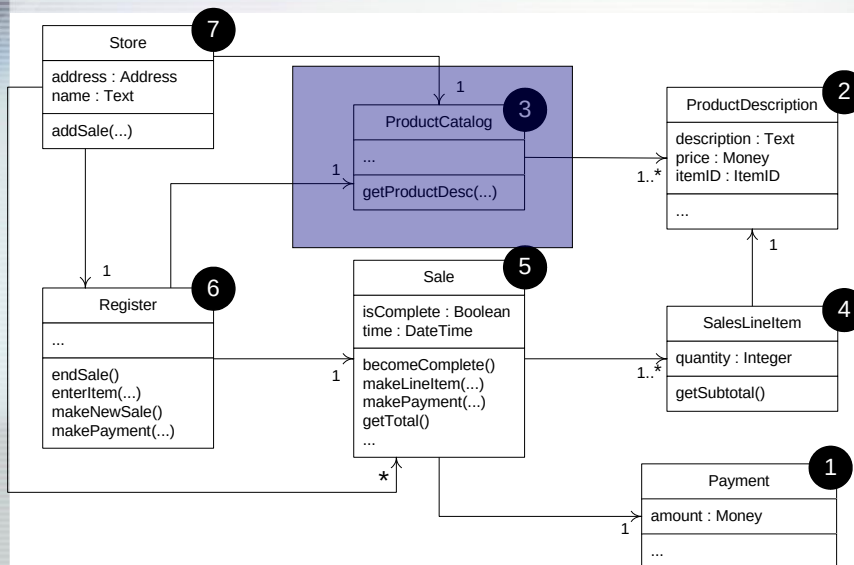
    public ItemID getItemID() { return id; }

    public Money getPrice() { return price; }
    public String getDescription() { return description; }
}
```

Copyright © 2020 by Dr. Simon Xu,

27

Order of Implementation



Copyright © 2020 by Dr. Simon Xu,

28

Class ProductCatalog

```
public class ProductCatalog
{
    private Map<ItemID, ProductDescription>
        descriptions = new HashMap<>(<ItemID, ProductDescription>);

    public ProductCatalog()
    {
        // sample data
        ItemID id1 = new ItemID( 100 );
        ItemID id2 = new ItemID( 200 );
        Money price = new Money( 3 );

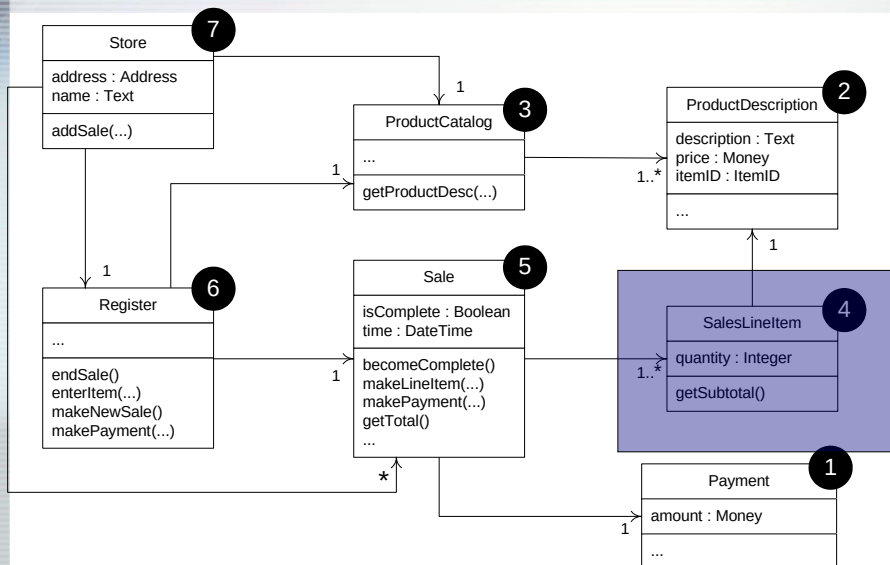
        ProductDescription desc;
        desc = new ProductDescription( id1, price, "product 1" );
        descriptions.put( id1, desc );
        desc = new ProductDescription( id2, price, "product 2" );
        descriptions.put( id2, desc );
    }

    public ProductDescription getProductDescription( ItemID id )
    {
        return descriptions.get( id );
    }
}
```

Copyright © 2020 by Dr. Simon Xu,

29

Order of Implementation



Copyright © 2020 by Dr. Simon Xu,

30

Class SalesLineItem

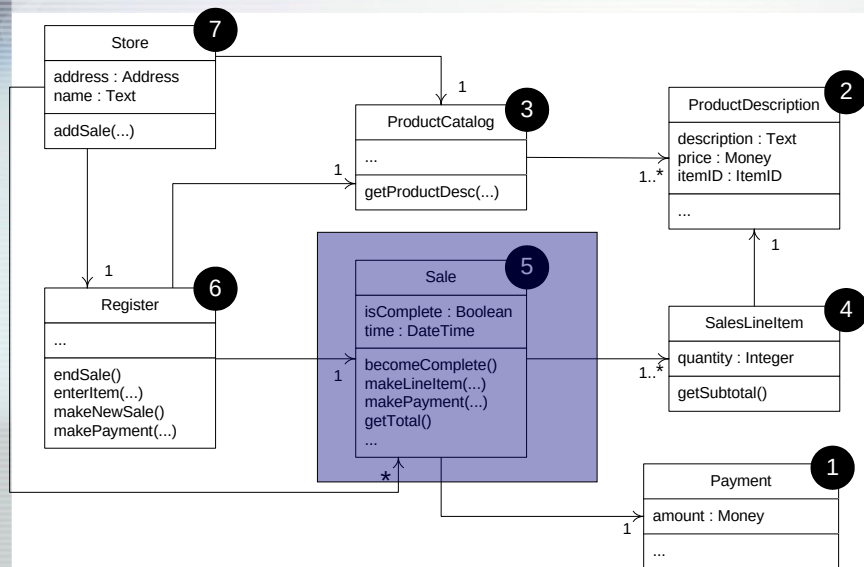
```
public class SalesLineItem
{
    private int    quantity;
    private    ProductDescription    description;

    public SalesLineItem (ProductDescription desc, int quantity )
    {
        this.description = desc;
        this.quantity = quantity;
    }
    public Money getSubtotal()
    {
        return description.getPrice().times( quantity );
    }
}
```

Copyright © 2020 by Dr. Simon Xu,
CC BY-SA 4.0

31

Order of Implementation



Copyright © 2020 by Dr. Simon Xu,
CC BY-SA 4.0

32

Class Sale

```

public class Sale
{
    [ ]

    public Money getBalance()
    {
        [ ]
    }

    public void becomeComplete() { isComplete = true; }
    public boolean isComplete() { return isComplete; }

    public void makeLineItem
        ( ProductDescription desc, int quantity )
    {
        [ ]
    }

    public Money getTotal()
    {
        Money total = new Money();
        Money subtotal = null;

        for ( SalesLineItem lineItem : lineItems )
        {
            subtotal = lineItem.getSubtotal();
            total.add( subtotal );
        }
        return total;
    }

    public void makePayment( Money cashTendered )
    {
        [ ]
    }
}

```

Copyright © 2020 by Dr. Simon Xu,

33

Class Sale

```

public class Sale
{
    private List<SalesLineItem> lineItems =
        new ArrayList<>(<SalesLineItem>);
    private Date date = new Date();
    private boolean isComplete = false;
    private Payment payment;

    public Money getBalance()
    {
        return payment.getAmount().minus( getTotal() );
    }

    public void becomeComplete() { isComplete = true; }
    public boolean isComplete() { return isComplete; }

    public void makeLineItem
        ( ProductDescription desc, int quantity )
    {
        lineItems.add( new SalesLineItem( desc, quantity ) );
    }

    public Money getTotal()
    {
        Money total = new Money();
        Money subtotal = null;

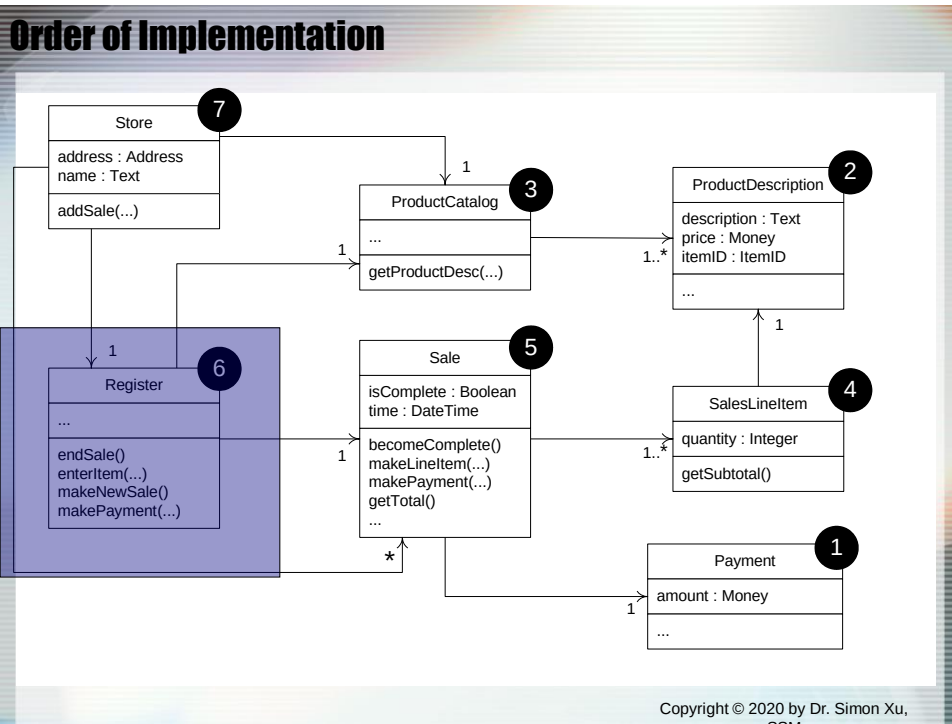
        for ( SalesLineItem lineItem : lineItems )
        {
            subtotal = lineItem.getSubtotal();
            total.add( subtotal );
        }
        return total;
    }

    public void makePayment( Money cashTendered )
    {
        payment = new Payment( cashTendered );
    }
}

```

Copyright © 2020 by Dr. Simon Xu,

34



35

Class Register

```

public class Register
{
    private ProductCatalog catalog;
    private Sale currentSale;

    public Register( ProductCatalog catalog )
    {
        this.catalog = catalog;
    }

    public void endSale()
    {
        
    }

    public void enterItem( ItemID id, int quantity )
    {
        
    }

    public void makeNewSale()
    {
        
    }

    public void makePayment( Money cashTendered )
    {
        
    }
}
  
```

Copyright © 2020 by Dr. Simon Xu, CMU

36

Class Register

```
public class Register
{
    private ProductCatalog catalog;
    private Sale currentSale;

    public Register( ProductCatalog catalog )
    {
        this.catalog = catalog;
    }

    public void endSale()
    {
        currentSale.becomeComplete();
    }

    public void enterItem( ItemID id, int quantity )
    {
        ProductDescription desc = catalog.getProductDescription( id );
        currentSale.makeLineItem( desc, quantity );
    }

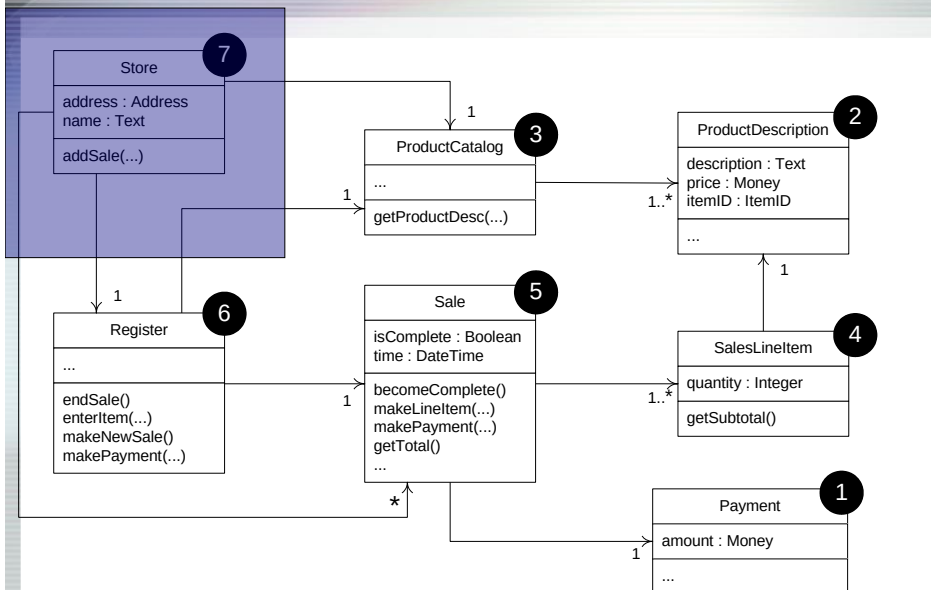
    public void makeNewSale()
    {
        currentSale = new Sale();
    }

    public void makePayment( Money cashTendered )
    {
        currentSale.makePayment( cashTendered );
    }
}
```

Copyright © 2020 by Dr. Simon Xu,
SCM

37

Order of Implementation



Copyright © 2020 by Dr. Simon Xu,
SCM

38

Class Store

```
public class Store
{
    
}
```

Copyright © 2020 by Dr. Simon Xu,
CC BY-SA

39

Class Store

```
public class Store
{
    private ProductCatalog catalog = new ProductCatalog();
    private Register register = new Register( catalog );

    public Register getRegister() { return register; }
}
```

Copyright © 2020 by Dr. Simon Xu,
CC BY-SA

40